

DOCUMENTATION TECHNIQUE COMPLÈTE · JUILLET 2026

SecureBackup-Chain

Architecture, flux de données, protocoles cryptographiques, consensus distribué et déploiement d'un système de sauvegarde décentralisé et auditable

Hyperledger

Fabric

v2.5.4 · Raft 3
orderers

IPFS

Cluster

Kubo + Raft
pinning

AES-256-

CBC

SHA-256 ·
IV par
fichier

Node.js 18

Express 4 ·
busboy stream

PostgreSQL

15

Prisma ORM

SSH2 ·

SFTP

ssh2-sftp-
client

ourolaongyassine@gmail.com · Version 1.0

Table des matières

1. Vue d'ensemble de l'architecture

- 1.1 Schéma global des composants
- 1.2 Séparation plan de contrôle / plan de données
- 1.3 Flux réseau et ports

2. Hyperledger Fabric — Architecture détaillée

- 2.1 Composants Fabric et leurs rôles
- 2.2 MSP et certificats X.509
- 2.3 Consensus Raft : fonctionnement interne
- 2.4 Cycle de vie d'une transaction
- 2.5 Chaincode backup-cc : logique métier
- 2.6 World State CouchDB et ledger

3. IPFS — Stockage distribué

- 3.1 Adressage par contenu (CID)
- 3.2 IPFS Cluster et consensus Raft
- 3.3 Interaction avec le backend

4. Pipeline cryptographique

- 4.1 AES-256-CBC et vecteur d'initialisation
- 4.2 Streaming single-pass (createEncryptStream)
- 4.3 Intégrité SHA-256
- 4.4 Gestion des clés

5. Backend Node.js — API REST

- 5.1 Architecture Express et middlewares
- 5.2 Authentification JWT et RBAC
- 5.3 Routes principales
- 5.4 Schéma de base de données PostgreSQL

6. Services SSH et SFTP

- 6.1 Différences SSH vs SFTP
- 6.2 Sécurité des transports distants
- 6.3 fetchRemotePath : fichier et répertoire

7. Planificateur de sauvegardes

- 7.1 node-cron et expressions cron

7.2

executeSchedule : pipeline complet

8. Flux de données complets

- | | |
|-----|--|
| 8.1 | Upload local (streaming) |
| 8.2 | Sauvegarde SFTP distante |
| 8.3 | Planification automatique |
| 8.4 | Restauration et vérification d'intégrité |

9. Sécurité multicouche

10. Déploiement Docker et réseau

Vue d'ensemble de l'architecture

SecureBackup-Chain sépare strictement le plan de contrôle (blockchain — qui a fait quoi et quand) du plan de données (IPFS — les octets chiffrés). Ces deux plans sont indépendants et se renforcent mutuellement.

1.1 Schéma global des composants

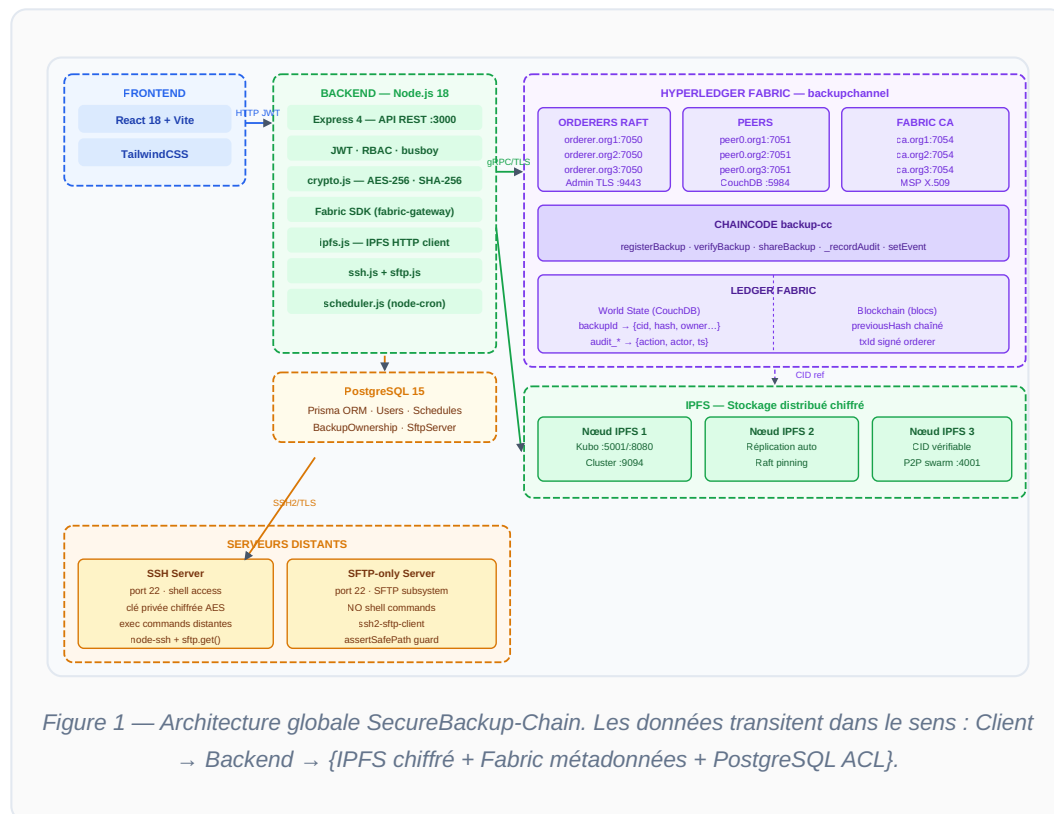


Figure 1 — Architecture globale SecureBackup-Chain. Les données transitent dans le sens : Client → Backend → {IPFS chiffré + Fabric métadonnées + PostgreSQL ACL}.

1.2 Séparation plan de contrôle / plan de données

Plan de contrôle — Fabric

- Qui a sauvegardé quoi et quand
- Hash SHA-256 du fichier clair
- CID IPFS (pointeur vers données)
- Identité X.509 du créateur
- Audit trail immuable
- Partages et accès

Plan de données — IPFS

- Octets chiffrés AES-256-CBC
- Adressés par CID (hash contenu)
- Réplication multi-nœuds
- Récupérable avec CID seul
- Illisible sans MASTER_KEY

Plan d'accès — PostgreSQL

- Utilisateurs et rôles
- ACL applicatifs (backupOwnership)
- Planifications cron
- Runs et historique
- Serveurs SSH/SFTP

Principe clé : La compromission d'IPFS révèle des octets chiffrés illisibles. La compromission de PostgreSQL révèle des métadonnées mais pas les données. Seule la combinaison Fabric + IPFS + MASTER_KEY permet une lecture complète — et cette combinaison n'existe que dans le backend en mémoire vive, jamais persistée ensemble.

1.3 Flux réseau et ports

SERVICE	PORT	PROTOCOLE	SENS	SÉCURITÉ
Frontend Vite	5173	HTTP	Client → Navigateur	Dev only
API Backend	3000	HTTP/REST	Client → Backend	JWT Bearer token
Orderer gRPC	7050	gRPC/TLS	Backend → Fabric	mTLS certificat X. 509
Peer gRPC	7051	gRPC/TLS	Backend → Fabric	mTLS certificat X. 509
Orderer Admin	9443	HTTPS	Admin → Orderer	TLS + admin cert
Fabric CA	7054	HTTPS	Admin → CA	TLS
CouchDB	5984	HTTP	Peer → CouchDB	Interne Docker
IPFS API	5001	HTTP	Backend → IPFS	Interne réseau
IPFS Gateway	8080	HTTP	Client → IPFS	Lecture publique (chiffré)
IPFS Swarm	4001	TCP/UDP	IPFS ↔ IPFS	Libp2p chiffré
IPFS Cluster	9094	HTTP	Backend → Cluster	Interne réseau
SSH/SFTP distant	22	SSH2	Backend → Serveur	Clé privée chiffrée

Hyperledger Fabric — Architecture détaillée

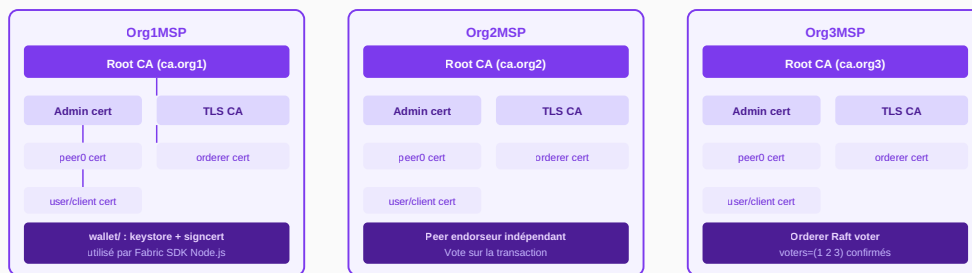
Fabric est le socle de confiance du système. Il garantit que chaque sauvegarde enregistrée est authentifiée, ordonnée, immuable et auditable — indépendamment de la disponibilité du backend ou de PostgreSQL.

2.1 Composants Fabric et leurs rôles

COMPOSANT	RÔLE	DANS SECUREBACKUP-CHAIN
Fabric CA	Autorité de certification — émet les identités X.509	3 CA (org1, org2, org3) — MSP par organisation
Orderer	Ordonne les transactions, forme les blocs, distribue	3 orderers Raft — backupchannel
Peer	Exécute le chaincode, maintient le ledger, valide les tx	1 peer par org (peer0.org1/2/3)
CouchDB	State database — world state JSON interrogeable	1 CouchDB par peer — requêtes Mango
Chaincode	Logique métier — s'exécute dans container Docker	backup-cc (Node.js) — version incrémentée
Channel	Réseau privé isolé entre organisations	backupchannel — Org1 + Org2 + Org3
MSP	Mappe certificats ↔ identités et rôles Fabric	1 MSP par org : Org1MSP, Org2MSP, Org3MSP

2.2 MSP et certificats X.509

Le **Membership Service Provider (MSP)** est la couche d'identité de Fabric. Chaque organisation dispose d'une hiérarchie de certificats :



Le wallet Node.js (./wallet/) stocke les identités des clients Fabric (paires clé privée + certificat signé). Le SDK fabric-gateway signe chaque transaction avec la clé privée du user avant envoi au peer.

2.3 Consensus Raft : fonctionnement interne

Figure 3 — Protocole Raft : AppendEntries, quorum, et élection de leader.

2.4 Cycle de vie d'une transaction Fabric

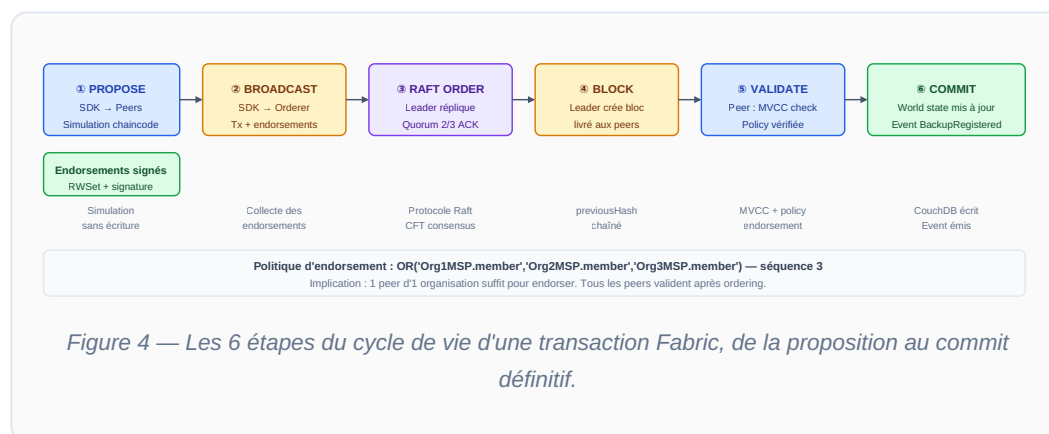


Figure 4 — Les 6 étapes du cycle de vie d'une transaction Fabric, de la proposition au commit définitif.

2.5 Chaincode backup-cc : logique métier

FONCTION CHAINCODE	PARAMÈTRES	ACTION SUR LE LEDGER
registerBackup	backupId, cid, fileName, fileHash, fileSize, mimeType	putState(backupId, entry) + _recordAudit + setEvent
getBackup	backupId	getState(backupId) → JSON
verifyBackup	backupId, fileHash	getState → compare hash → retourne booléen
shareBackup	backupId, targetUser	putState (sharedWith.push) + _recordAudit
deleteBackup	backupId	putState (status='DELETED') + _recordAudit
queryBackupsByOwner	ownerId	getQueryResult (CouchDB Mango selector)
getAuditHistory	—	getQueryResult (audit_* keys, range scan)
recordAudit	action, target, details	putState("audit{ts}_{txId}", auditEntry)

Idempotence : registerBackup vérifie getState(backupId) avant écriture. Si la clé existe déjà, une erreur est levée. Cela protège contre les doubles inscriptions en cas de retry réseau du SDK.

Structure d'une entrée backup dans le World State

World State CouchDB — clé : backupId

JSON

```
{
  "backupId": "uuid-v4",
  "cid": "QmXoypiz...", // CID IPFS du fichier chiffré
  "fileName": "backup_2026-07-10.tar.gz",
  "fileHash": "sha256:a3f2...", // hash du CLAIR avant chiffrement
  "fileSize": 1048576, // octets du clair
  "mimeType": "application/gzip",
  "ownerId": "x509:/C=FR/O=Org1/CN=user1", // identité X.509
  "ownerMSP": "Org1MSP",
  "timestamp": "2026-07-10T10:30:00.000Z", // depuis tx header
  "txId": "a1b2c3...", // txId Fabric unique
  "status": "ACTIVE", // ACTIVE | DELETED
  "sharedWith": [] // liste des partages
}
```

2.6 World State CouchDB et ledger

World State (CouchDB)

- État courant de chaque clé
- Interrogeable par Mango query
- Mis à jour à chaque commit
- Reconstructible depuis le ledger
- 1 CouchDB par peer (isolé)

Ledger (Blockchain)

- Séquence immuable de blocs
- Chaque bloc : previousHash
- Chaque tx signée par orderer
- Non effaçable, non modifiable
- Historique complet de toutes les tx

IPFS — Stockage distribué

IPFS stocke les fichiers chiffrés et les adresse par leur empreinte cryptographique (CID). La perte d'un nœud n'entraîne aucune perte de données. Un fichier modifié obtient un CID différent — la falsification est structurellement impossible.

3.1 Adressage par contenu (CID)

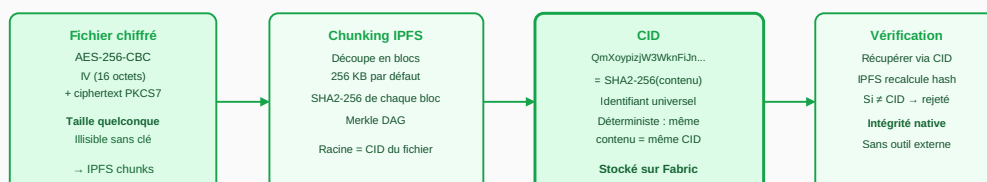


Figure 5 — De l'octet chiffré au CID : chunking, Merkle DAG, et vérification native par IPFS.

Propriété fondamentale : Il est cryptographiquement impossible de modifier le contenu d'un fichier IPFS sans changer son CID. Comme le CID est stocké sur le ledger Fabric (immuable), toute falsification du fichier IPFS est détectable instantanément.

3.2 IPFS Cluster et consensus Raft

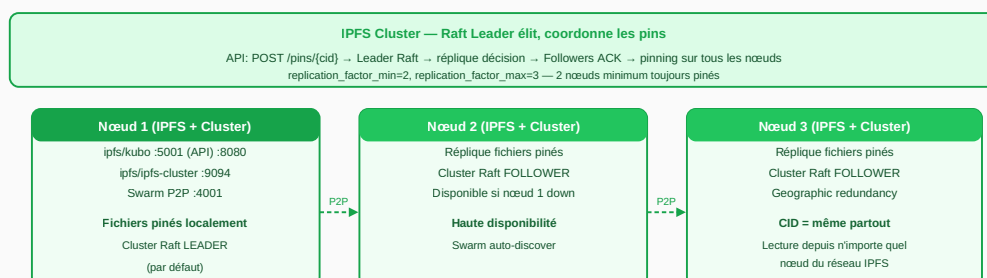


Figure 6 — IPFS Cluster : coordination Raft pour les pins, réplification automatique sur 2-3 nœuds, haute disponibilité.

3.3 Interaction avec le backend (ipfs.js)

backend/src/services/ipfs.js – méthodes clés

IPFS HTTP API

```
// Connexion au cluster IPFS via HTTP
const client = create({ url: env.IPFS_API_URL }); // http://ipfs-node:5001

// ① Upload depuis async iterable (streaming)
async function addFromAsyncIterable(asyncIterable, fileName) {
  const result = await client.add(
    { content: asyncIterable, path: fileName },
    { pin: true, cidVersion: 0 } // pin immédiat, CIDv0
  );
  return result.cid.toString(); // ex: "QmXoybiz..."
}

// ② Récupération par CID pour restauration
async function getFile(cid) {
  const chunks = [];
  for await (const chunk of client.cat(cid)) {
    chunks.push(chunk);
  }
  return Buffer.concat(chunks); // buffer chiffré
}

// ③ Pin via IPFS Cluster pour réplication garantie
async function pinToCluster(cid) {
  await fetch(`${env.IPFS_CLUSTER_URL}/pins/${cid}`,
    { method: 'POST' }); // cluster réplique sur N nœuds
}
```

NOTES

① addFromAsyncIterable

Accepte un générateur async — les chunks chiffrés sont streamés sans buffer complet. IPFS chunke lui-même le flux entrant.

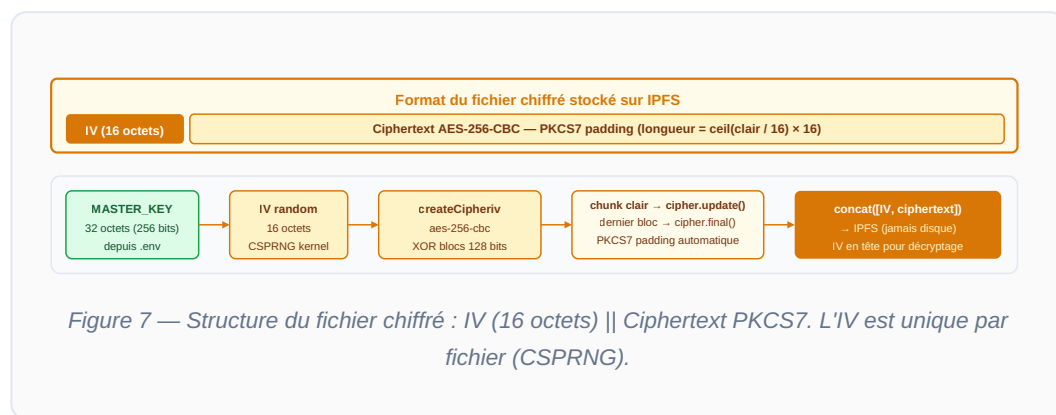
② getFile (restauration)

ipfs.cat() retourne un async iterable de chunks. On concatène pour obtenir le buffer chiffré complet, puis decryptAES().

Pipeline cryptographique AES-256-CBC

Tout fichier est chiffré avant de quitter la mémoire Node.js. Le hash SHA-256 du fichier clair est calculé simultanément en un seul passage. Aucune donnée claire n'est jamais écrite sur disque ou envoyée sur le réseau.

4.1 AES-256-CBC et vecteur d'initialisation



PARAMÈTRE AES	VALEUR	RAISON
Algorithme	AES-256-CBC	Standard NIST, 256 bits = résistant aux attaques brute-force jusqu'en 2100+
Taille de clé	256 bits (32 octets)	MASTER_KEY hex 64 chars → Buffer.from(key,'hex')
IV	16 octets aléatoires par fichier	IND-CPA sécurité sémantique — deux fichiers identiques → chiffrés différents
Mode	CBC (Cipher Block Chaining)	Chaque bloc XORé avec le précédent — diffusion des erreurs
Padding	PKCS7 automatique	Node.js crypto gère automatiquement le padding du dernier bloc

4.2 Streaming single-pass — createEncryptStream

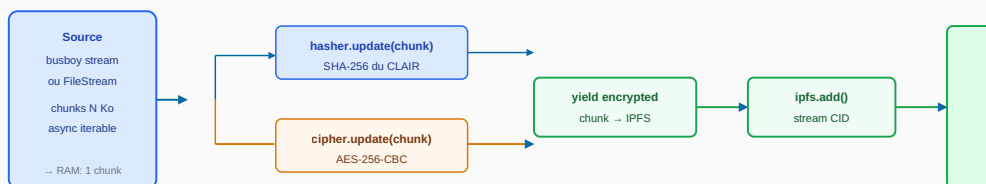


Figure 8 — createEncryptStream : hash SHA-256 et chiffrement AES en parallèle sur le même chunk. Un seul passage mémoire.

backend/src/services/crypto.js — createEncryptStream

single-pass

```

function createEncryptStream(inputAsyncIterable, hexKey) {
  const key = Buffer.from(hexKey, 'hex');
  const iv = crypto.randomBytes(16); // IV unique par appel
  const cipher = crypto.createCipheriv('aes-256-cbc', key, iv);
  const hasher = crypto.createHash('sha256'); // hash du CLAIR
  let size = 0;

  async function* gen() {
    yield iv; // premier chunk = IV
    for await (const chunk of inputAsyncIterable) {
      hasher.update(chunk); // hash avant chiffrement
      size += chunk.length;
      const encrypted = cipher.update(chunk);
      if (encrypted.length > 0) yield encrypted; // yield immédiat
    }
    const final = cipher.final(); // PKCS7 padding
    if (final.length > 0) yield final;
  } // getHash() valide APRÈS fi

  return { stream: gen(), getHash: () => hasher.digest('hex'), getSize: () => size };
}
  
```

4.3 Intégrité SHA-256 — double vérification

NIVEAU	HASH	CE QU'IL PROUVE	STOCKÉ DANS
Niveau 1	CID IPFS = SHA2-256(chiffré)	Le fichier chiffré n'a pas été modifié sur IPFS	IPFS (implicite) + Fabric
Niveau 2	fileHash = SHA-256(clair)	Le fichier déchiffré est identique à l'original	Fabric ledger

La vérification complète combine les deux niveaux : récupérer depuis IPFS (CID vérifié) → déchiffrer → SHA-256 → comparer avec fileHash sur Fabric.

4.4 Gestion des clés

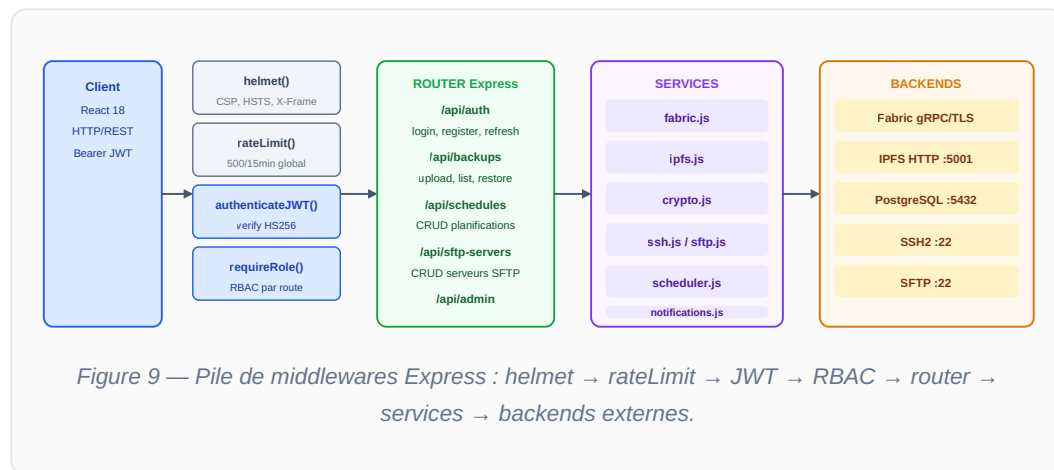
CLÉ	USAGE	STOCKAGE	ROTATION
MASTER_KEY	Chiffrement AES des fichiers	Variable d'env (.env non commité)	Nécessite re-chiffrement de tous les fichiers
JWT_SECRET	Signature des tokens JWT	Variable d'env	Invalide tous les tokens existants
Clés SSH privées	Connexion aux serveurs distants	DB PostgreSQL chiffrées AES-GCM (MASTER_KEY)	Par serveur SSH
Fabric privkeys	Signature des transactions Fabric	wallet/ (hors .gitignore)	Via Fabric CA (enroll)

Point critique : MASTER_KEY est la clé maîtresse du système. Sa perte rend tous les fichiers sur IPFS illisibles. Sa compromission permet le déchiffrement de tous les fichiers. Elle doit être sauvegardée dans un HSM ou un coffre-fort de secrets (Vault) en production.

Backend Node.js — API REST

L'API REST est le point d'entrée unique du système. Elle orchestre tous les services — Fabric, IPFS, PostgreSQL, SSH/SFTP — et assure que les invariants de sécurité sont respectés à chaque requête.

5.1 Architecture Express et middlewares



5.2 Authentification JWT et RBAC

backend/src/middleware/auth.js – middlewares clés

JWT HS256

RBAC

```
// ① Vérification JWT sur chaque requête protégée
function authenticateJWT(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Token requis' });
  try {
    req.user = jwt.verify(token, env.JWT_SECRET); // { sub, role, exp }
    next();
  } catch {
    res.status(401).json({ error: 'Token invalide ou expiré' });
  }
}

// ② Contrôle d'accès basé sur les rôles
function requireRole(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user?.role))
      return res.status(403).json({ error: 'Accès refusé' });
    next();
  };
}

// Usage : protège une route admin seulement
router.delete('/users/:id', authenticateJWT, requireRole('admin'), handler)
```

5.3 Routes principales

ROUTE	MÉTHODE	RÔLES	DESCRIPTION
/api/auth/login	POST	Public	Authentification → JWT + refresh token
/api/backups	POST	admin, responsable	Upload local streaming — busboy + AES + IPFS + Fabric
/api/backups	GET	Tous	Liste des sauvegardes (filtrée par rôle)
/api/backups/:id/restore	POST	admin, responsable	Récupère IPFS → déchiffre → stream client
/api/backups/:id/verify	GET	Tous	Récupère hash Fabric → recalcule → compare
	POST		

/api/backups/sftp-remote		admin, responsable	Sauvegarde depuis serveur SFTP distant
/api/sftp-servers	GET/ POST	admin, responsable	CRUD serveurs SFTP (credentials chiffrés)
/api/schedules	GET/ POST	admin, responsable	CRUD planifications SSH/SFTP
/api/schedules/:id/run	POST	admin	Déclenche manuellement une planification
/api/admin/export-config	GET	admin	Export config chiffrée AES-256-GCM (.tar.gz.enc)
/api/admin/import-config	POST	admin	Import et validation de configuration

5.4 Schéma de base de données PostgreSQL

```
// Utilisateurs et rôles
model User {
  id          String   @id @default(uuid())
  email       String   @unique
  password    String   // bcrypt hash
  role        String   // admin | responsable | auditeur
  createdAt   DateTime @default(now())
  ownedBackups BackupOwnership[]
  schedules   ScheduledBackup[]
}

// Propriété des sauvegardes (ACL applicatif)
model BackupOwnership {
  id          String   @id @default(uuid())
  backupId    String   // = backupId Fabric
  userId      String
  user        User     @relation(fields: [userId], references: [id])
}

// Serveurs SFTP – credentials chiffrés
model SftpServer {
  id          String   @id @default(uuid())
  name        String
  host        String
  port        Int      @default(22)
  username    String
  auth_type   String   // password | key
  encryptedCredentials String // JSON chiffré AES-256-GCM
  schedules   ScheduledBackup[]
}

// Planifications automatiques
model ScheduledBackup {
  id          String   @id @default(uuid())
  name        String
  cronExpression String // ex: "0 2 * * *"
  remotePath   String
  status       String   // active | paused
  sshServerId  String?   // NULL si SFTP
  sftpServerId String?   // NULL si SSH
  ownerId      String
  retentionDays Int      @default(30)
  runs         ScheduledBackupRun[]
}

// Historique des exécutions
model ScheduledBackupRun {
  id          String   @id @default(uuid())
  scheduleId  String
  status       String   // running | success | error
  startedAt   DateTime
  completedAt DateTime?
  backupId    String?   // backupId Fabric si succès
}
```

```
    errorMessage String?  
}
```

Services SSH et SFTP

Le système supporte deux modes de connexion aux serveurs distants : SSH (avec shell — `node-ssh`) et SFTP (sans shell — `ssh2-sftp-client`). Chaque mode offre des garanties de sécurité différentes.

6.1 Différences SSH vs SFTP

SSH (node-ssh)	SFTP (ssh2-sftp-client) ✓ Recommandé
• Protocole SSH2 complet — canal shell ouvert • Exécute des commandes distantes (exec) • Transfert fichiers via SFTP subsystem intégré • Compatible avec tous les serveurs SSH standard • Surface d'attaque : injection de commandes possible • Usage : serveurs avec accès shell complet ⚠ Risque : commandes arbitraires si path non valide	• Sous-protocole SSH2 — PAS de shell ouvert • Uniquement des opérations fichier (get, put, ls...) • Compatible serveurs SFTP-only (chroot jail) • Aucune commande shell exécutée à distance • <code>assertSafePath()</code> filtre les chemins interdits • <code>sftp.exists()</code> → 'd' '.' false (sans ls) ✓ Surface d'attaque minimale — recommandé

Figure 10 — Comparaison SSH (shell complet) vs SFTP (sous-système fichier uniquement).

6.2 Sécurité des transports distants

MENACE	CONTRE-MESURE	CODE
Path traversal (../.. etc/passwd)	<code>path.posix.normalize()</code> avant contrôle liste noire	<code>assertSafePath()</code>
Lecture de fichiers système sensibles	FORBIDDEN_PATHS : /etc, /root, /proc, /sys, /dev	<code>sftp.js</code> ligne 9
Saturation disque (fichier énorme)	MAX_SIZE_BYTES = 10 Go — vérifié par <code>sftp.stat()</code>	<code>sftp.js</code> ligne 10
Credentials en clair en DB	Chiffrés AES-256-GCM avec MASTER_KEY avant stockage	<code>scheduler.js</code> <code>decryptCredentials()</code>
Commandes shell arbitraires	SFTP uniquement pour les planifications — pas de <code>exec()</code>	<code>sftp.js</code> — pas de <code>ssh.execCommand()</code>
Fuite de socket en cas d'erreur	<code>try/catch/finally</code> — <code>closeConnection()</code> garanti	<code>scheduler.js</code> finally block

6.3 fetchRemotePath : gestion fichier et répertoire

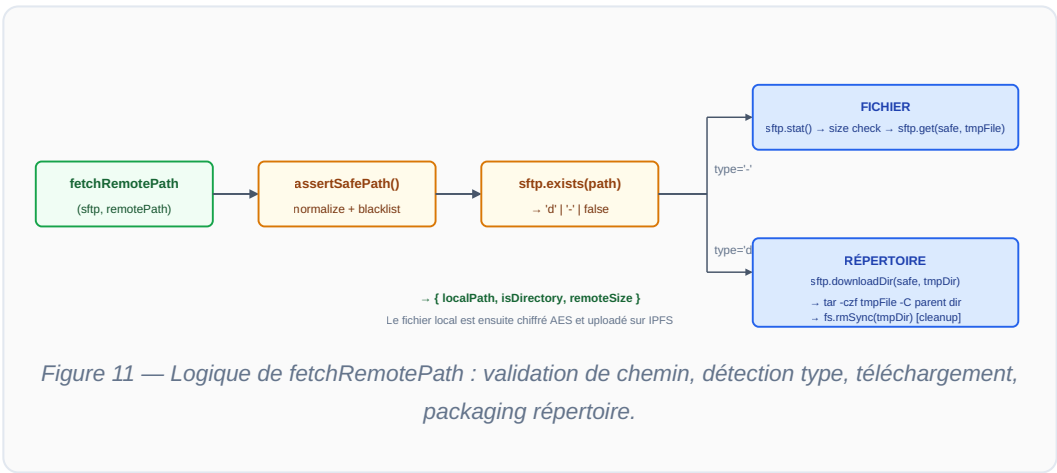


Figure 11 — Logique de `fetchRemotePath` : validation de chemin, détection type, téléchargement, packaging répertoire.

Planificateur de sauvegardes

Le planificateur exécute automatiquement des sauvegardes SSH ou SFTP selon des expressions cron configurables. Il partage exactement le même pipeline cryptographique que les routes API manuelles.

7.1 node-cron et expressions cron

EXPRESSION	SIGNIFICATION	USAGE TYPIQUE
<code>0 2 * * *</code>	Tous les jours à 02:00	Sauvegarde nightly
<code>0 */6 * * *</code>	Toutes les 6 heures	Sauvegarde fréquente
<code>0 2 * * 0</code>	Dimanche à 02:00	Sauvegarde hebdomadaire
<code>0 0 1 * *</code>	1er du mois à minuit	Sauvegarde mensuelle
<code>* /30 * * * *</code>	Toutes les 30 minutes	Sauvegarde quasi-temps-réel

registerTask() : Si une tâche pour ce `scheduleId` existe déjà dans `activeTasks`, elle est arrêtée avant enregistrement de la nouvelle. Cela évite les doublons lors d'une modification de planification à chaud.

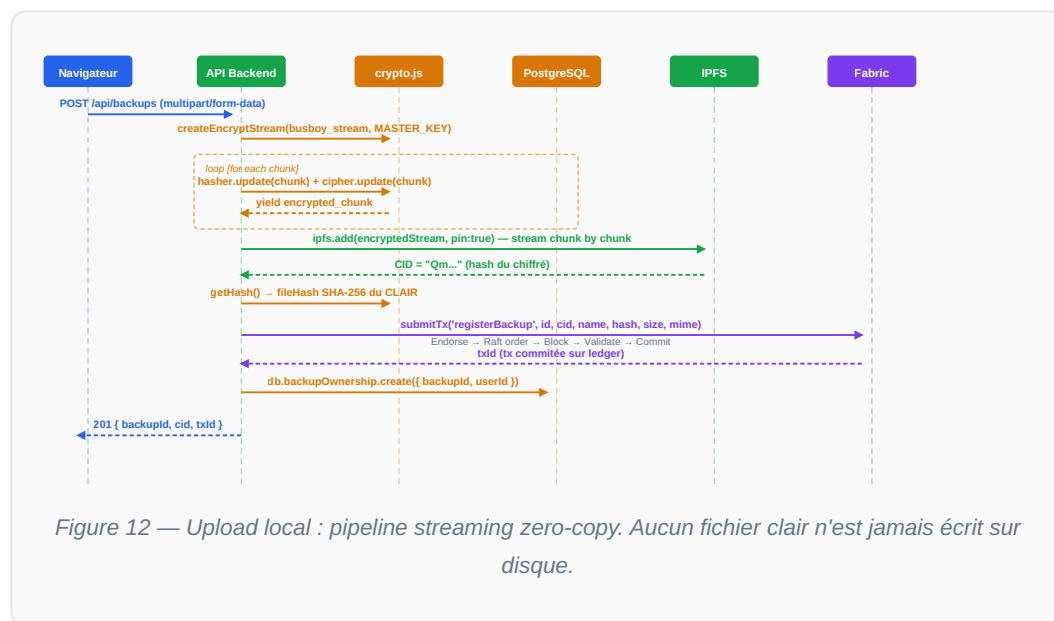
7.2 executeSchedule — pipeline complet

- 1 **Chargement** : `db.scheduledBackup.findUnique({ include: { sshServer, sftpServer } })` — une seule requête Prisma. Vérifie `status === 'active'`.
- 2 **Run "running"** : `db.scheduledBackupRun.create({ status: 'running', startedAt: new Date() })` avant tout traitement. Les runs bloqués (status=running depuis trop longtemps) sont détectables.
- 3 **Détection protocole** : `useSftp = !!schedule.sftpServerId`. Un seul des deux champs est non-null (contrainte Prisma). `svc = useSftp ? sftpService : sshService`.
- 4 **Déchiffrement credentials** : `JSON.parse(decryptCredentials(server.encryptedCredentials))` — les credentials ne sont déchiffrés qu'en mémoire vive, jamais écrits.
- 5 **Connexion + download** : `svc.connect(params)` puis `svc.fetchRemotePath(conn, remotePath) → { localPath }`. La connexion est fermée immédiatement après.
- 6 **Pipeline crypto** : `buf = fs.readFileSync(localPath) → encryptAES(buf, MASTER_KEY) → ipfs.add(enc) → CID`. Fichier tmp supprimé après.
- 7 **Inscription Fabric** : `fabric.submitTransaction('registerBackup', id, cid, name, sha256(buf), size, mime) → txId`. Attend confirmation Raft.
- 8 **Mise à jour run** : `updateRun({ status: 'success', completedAt, backupId })` ou `{ status: 'error', errorMessage }` en cas d'échec.
- 9 **Notification** : `notify(ownerId, 'schedule_success', ...)` via WebSocket ou email. Les admins sont notifiés en cas d'erreur.

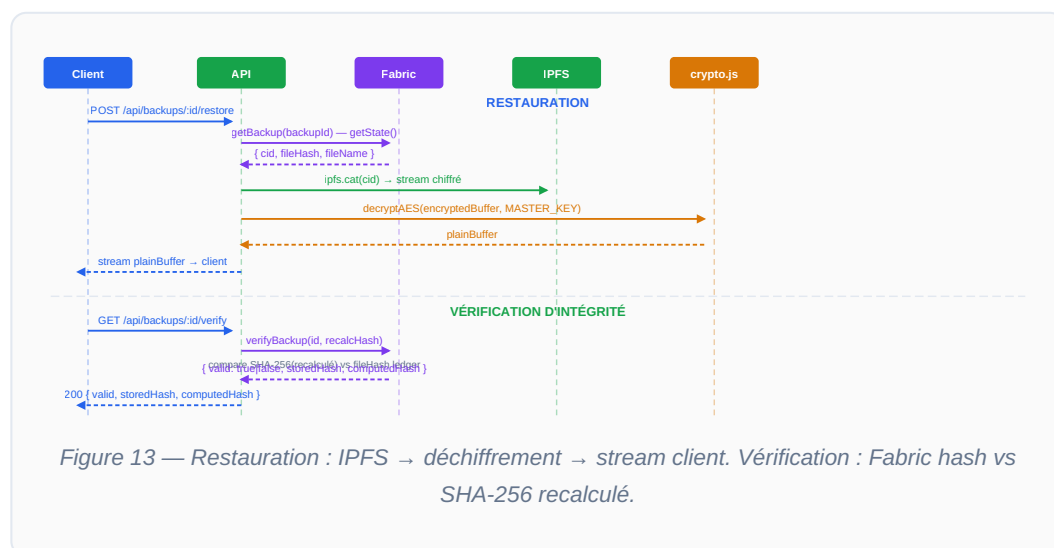
Flux de données complets

Quatre flux couvrent toutes les opérations principales du système. Chacun montre précisément comment les données transitent à travers chaque couche, avec les transformations appliquées.

8.1 Upload local (streaming zero-copy)



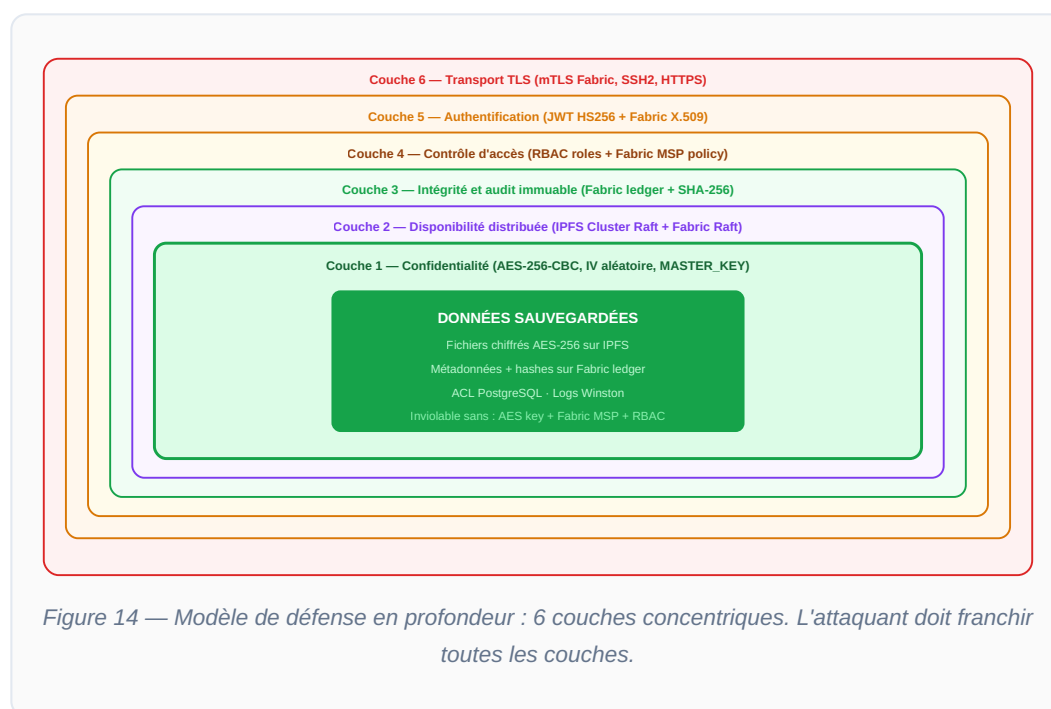
8.2 Restauration et vérification d'intégrité



Sécurité multicouche

La sécurité de SecureBackup-Chain repose sur 6 couches indépendantes. La compromission d'une couche ne suffit pas à compromettre le système dans son ensemble.

9.1 Les 6 couches de défense



COUCHE	MÉCANISME	RÉSISTE À
1 — Confidentialité	AES-256-CBC + IV aléatoire + MASTER_KEY	Vol des fichiers IPFS, vol du disque
2 — Disponibilité	IPFS Cluster Raft + Fabric Raft 3 orderers	Panne d'un nœud IPFS ou orderer
3 — Intégrité	SHA-256 sur ledger Fabric + CID IPFS	Falsification de fichier, audit frauduleux
4 — Contrôle d'accès	RBAC Express + MSP Policy chaincode	Élévation de privilèges, accès non autorisé
5 — Authentification	JWT HS256 + certificats X.509 Fabric	Usurpation d'identité, accès API sans auth
6 — Transport		

mTLS Fabric + SSH2 + HTTPS
+ rate limiting

Écoute réseau, man-in-the-
middle, brute-force

9.2 Analyse des vecteurs d'attaque

VECTEUR D'ATTAQUE	COUCHES CONCERNÉES	RÉSULTAT POUR L'ATTAQUANT	DÉFENSE RESTANTE
Vol d'un nœud IPFS	Couche 1	Octets chiffrés illisibles	AES-256 tient
Dump PostgreSQL	Couches 4,5	Métadonnées + CIDs (pas les données)	AES sur IPFS tient
JWT volé (XSS)	Couche 5	Accès API rôle victime uniquement	Données IPFS chiffrées, ledger intact
MASTER_KEY compromise	Couche 1	Déchiffrement de tous les fichiers	Audit Fabric, rotation clé nécessaire
Peer Fabric corrompu	Couche 3	Ne peut pas modifier le ledger seul	Politique endorsement + Raft quorum
Admin malveillant	Couches 4,3	Peut créer des sauvegardes mais pas effacer l'audit	Ledger Fabric immuable
Path traversal SFTP	Couche 6	Bloqué par <code>assertSafePath()</code>	<code>FORBIDDEN_PATHS + normalize()</code>
DDoS API	Couche 6	Rate limiting 500 req/15min	<code>helmet()</code> + <code>express-rate-limit</code>

Déploiement Docker et réseau

L'ensemble du système se déploie via Docker Compose. Les composants Fabric, IPFS, backend et frontend s'exécutent dans des conteneurs isolés sur un réseau bridge dédié.

10.1 Topologie Docker Compose

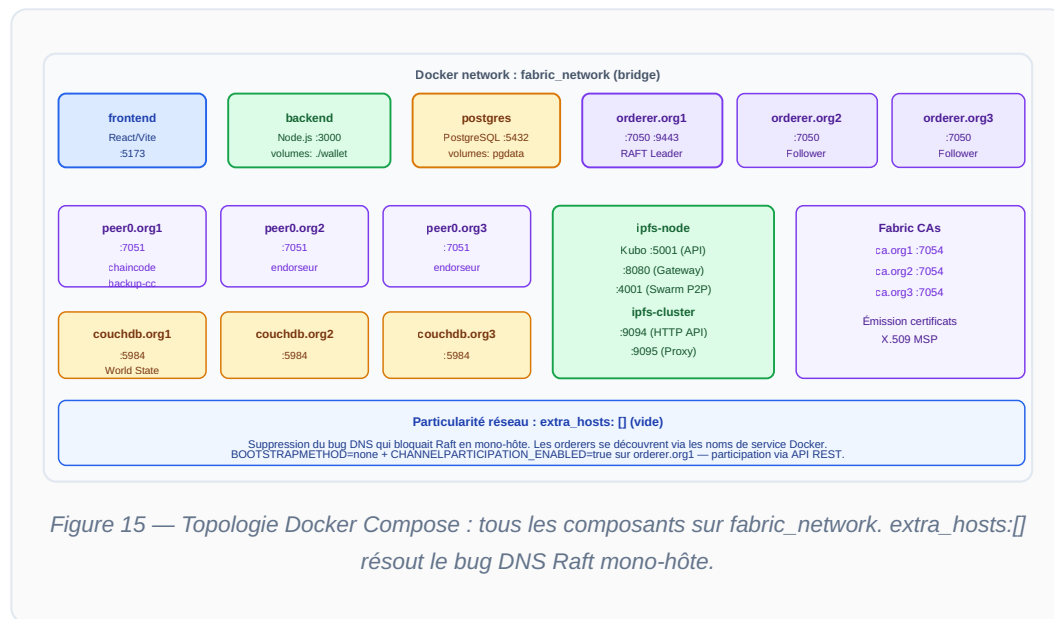


Figure 15 — Topologie Docker Compose : tous les composants sur `fabric_network`. `extra_hosts:[]` résout le bug DNS Raft mono-hôte.

10.2 Variables d'environnement critiques

`.env.example` – variables principales

Ne jamais commiter

```
# Sécurité API
NODE_ENV=production
API_PORT=3000
JWT_SECRET=<génére : openssl rand -hex 64>

# Clé maître de chiffrement AES-256
MASTER_KEY=<32 octets hex : openssl rand -hex 32>

# Fabric
FABRIC_CONNECTION_PROFILE=./config/connection-org1.json
FABRIC_WALLET_PATH=./wallet
FABRIC_CHANNEL=backupchannel
FABRIC_CHAINCODE=backup-cc
FABRIC_ORG_MSP=Org1MSP
FABRIC_USER=appUser

# IPFS
IPFS_API_URL=http://ipfs-node:5001
IPFS_CLUSTER_URL=http://ipfs-cluster:9094

# Base de données
DATABASE_URL=postgresql://user:pass@postgres:5432/securebackup

# Email (notifications)
SMTP_HOST=smtp.example.com
SMTP_PORT=587
SMTP_USER=notifications@example.com
SMTP_PASS=<mot de passe app>
```

10.3 Commandes de déploiement

COMMANDE	ACTION
<code>./scripts/start-network.sh</code>	Démarre le réseau Fabric (CAs, orderers, peers), crée le canal backupchannel, joint les orderers Raft
<code>./scripts/deploy-chaincode.sh</code>	Package, installe, approuve et commite backup-cc sur tous les peers (version + séquence incrémentées)
<code>docker compose up -d</code>	Démarre l'écosystème complet (backend, frontend, IPFS, PostgreSQL)
<code>node scripts/node-deployer.js</code>	Déploie dynamiquement un nouveau nœud distant (SSH) dans le canal Fabric existant

**`./scripts/stop-
network.sh`**

Arrête proprement tous les conteneurs Fabric, nettoie les volumes si demandé

Ordre impératif node-deployer.js : `start` → `orderjoin` → `raft`. La jointure de l'orderer au canal (`orderjoin` via participation API) doit précéder son ajout aux voters Raft. Inverser cet ordre cause un blocage du cluster.

Glossaire technique

TERME	DÉFINITION
AES-256-CBC	Advanced Encryption Standard, clé 256 bits, mode Cipher Block Chaining. Standard NIST pour le chiffrement symétrique.
backupchannel	Canal Fabric privé regroupant Org1, Org2 et Org3. Toutes les transactions de sauvegarde sont inscrites sur ce canal.
busboy	Parser multipart Node.js en streaming. Permet de traiter les uploads de fichiers chunk par chunk sans buffer complet en RAM.
CFT (Crash Fault Tolerant)	Propriété d'un système tolérant les pannes par crash (arrêt), mais pas les nœuds byzantins (malveillants). Raft est CFT.
CID	Content Identifier. Hash SHA2-256 du contenu d'un fichier IPFS, encodé en base58. Adresse unique et vérifiable.
CouchDB	Base de données JSON utilisée comme World State par les peers Fabric. Supporte les requêtes Mango (JSON query).
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator. Sur Linux : /dev/urandom — utilisé pour générer les IVs AES.
gRPC/TLS	Protocole de communication entre les composants Fabric (SDK ↔ peer, SDK ↔ orderer). TLS mutuel (mTLS) — les deux parties s'authentifient.
IND-CPA	Indistinguishability under Chosen Plaintext Attack. Propriété de sécurité sémantique assurée par l'IV aléatoire par fichier.
IPFS Cluster	Couche d'orchestration au-dessus d'IPFS Kubo. Coordonne les pins (réplication) entre nœuds via Raft.
IV (Vecteur d'initialisation)	16 octets aléatoires générés par fichier pour AES-CBC. Garantit que deux fichiers identiques donnent des chiffrés différents.
Ledger Fabric	Registre immuable composé de la blockchain (blocs ordonnés) et du World State (état courant). Stocké sur chaque peer.
MASTER_KEY	Clé de chiffrement AES-256 (32 octets hex). Secrète, jamais stockée sur IPFS ou Fabric. Depuis variable d'environnement.
Merkle DAG	Structure de données IPFS : graphe acyclique dirigé de hashes. Chaque nœud est identifié par le hash de son contenu.

MSP	Membership Service Provider. Composant Fabric qui traduit certificats X.509 en identités avec rôles et permissions.
mTLS	Mutual TLS. Les deux parties (client et serveur) s'authentifient mutuellement via certificats X.509. Utilisé par Fabric.
MVCC	Multi-Version Concurrency Control. Détecte les conflits de transactions concurrentes sur le même objet dans Fabric.
node-cron	Bibliothèque Node.js pour exécuter des tâches selon des expressions cron (planification horaire/quotidienne/etc.).
PKCS7	Standard de padding pour AES-CBC. Le dernier bloc est complété jusqu'à 16 octets avec des octets dont la valeur = nombre d'octets ajoutés.
Prisma ORM	ORM TypeScript/JavaScript pour PostgreSQL. Génère les migrations, valide le schéma et type-safe les requêtes.
Raft	Algorithme de consensus distribué Crash Fault Tolerant. Élit un leader, réplique le log, garantit l'ordre total des entrées.
RBAC	Role-Based Access Control. Les droits sont accordés par rôle (admin, responsable, auditeur), pas par utilisateur individuel.
RWSet	Read-Write Set. Ensemble des clés lues et modifiées par un chaincode lors d'une simulation. Vérifié lors du commit (MVCC).
SFTP subsystem	Sous-protocole SSH2 pour le transfert de fichiers. Ne nécessite pas d'accès shell — surface d'attaque minimale.
SHA-256	Secure Hash Algorithm 256 bits. Utilisé pour l'intégrité des fichiers (hash du clair stocké sur Fabric) et par IPFS (CID).
World State	Vue courante du ledger Fabric (clé → valeur JSON), stockée dans CouchDB. Mise à jour à chaque bloc commité.
X.509	Standard de certificat numérique PKI. Utilisé par Fabric CA pour identifier les peers, orderers et clients.